MultXeu: Mathematical Framework for Multicomponent Eutectic Cascade Modeling

Alan A. Smith*

August 25, 2026

Abstract

This document brings together the mathematical foundations, solver analysis, cascade logic, and experimental-error models used in the MultXeu framework. It combines the core thermodynamic derivations, numerical robustness studies, composition-evolution analysis, solvent-volume estimates, and the error model into a single consistent manuscript.

Contents

*www.cryss.co.uk

1 Generalized eutectic removal cascade

For the Python implementation of the conceptual algorithm puiblished in Tetrahedron Asymmetry 9 (1998) 2925–2937, consider a mixture with component mole fractions

$$X = (X_1, X_2, \ldots, X_n),$$

the eutectic cascade proceeds by repeatedly computing the eutectic composition at the eutectic temperature T_k, removing one dominant component per node, and renormalizing the remaining mixture.

1.1 Eutectic composition

For the current mixture X, solve for the eutectic temperature T_k and corresponding eutectic composition

$$X_{\text{eu}} = (X_{\text{eu},1}, \ldots, X_{\text{eu},n}).$$

1.2 Ratio calculation

For each active component, compute

$$r_i = \frac{X_{\text{eu},i}}{X_i}.$$

The dominant component is

$$i^* = \arg\max_i r_i, \qquad \lambda = r_{i^*}.$$

1.3 Scaling step

Scale the mixture:

$$X_i^{(\text{scaled})} = \lambda\, X_i, \qquad S_{\text{scaled}} = \sum_{i=1}^{n} X_i^{(\text{scaled})}.$$

1.4 Eutectic subtraction

Subtract the eutectic on the same scale:

$$X_i^{(\text{sub})} = X_i^{(\text{scaled})} - X_{\text{eu},i}.$$

By construction,

$$X_{i^*}^{(\text{sub})} = 0, \qquad X_i^{(\text{sub})} \geq 0.$$

Define

$$S_{\text{new}} = \sum_{i=1}^{n} X_i^{(\text{sub})}.$$

1.5 Step recovery

$$R_k = \frac{S_{\text{new}}}{S_{\text{scaled}}}.$$

Cumulative recovery after m nodes:

$$R_{\max} = \prod_{k=1}^{m} R_k.$$

1.6 Renormalization

$$X_i^{(\text{next})} = \frac{X_i^{(\text{sub})}}{S_{\text{new}}}.$$

1.7 Termination

Stop when the number of active components

$$\#\{i : X_i^{(\text{next})} > \varepsilon\}$$

is less than or equal to one.

2 Solving the n-dimensional eutectic

For each component i, the equilibrium liquid mole fraction follows the van't Hoff relation:

$$\ln(x_i) = \frac{\Delta_{\text{fus}} H_i}{R} \left(\frac{1}{T_{m,i}} - \frac{1}{T} \right).$$

Define:

$$C_i = \frac{\Delta_{\text{fus}} H_i}{R}, \qquad Q_i(T) = C_i \left(\frac{1}{T_{m,i}} - \frac{1}{T} \right), \qquad \beta_i = e^{Q_i(T)}.$$

Thus the SVL solubility is:

$$x_i(T) = \beta_i.$$

2.1 SVL model

$$F_i^{\text{SVL}}(T) = \beta_i.$$

First derivative

$$F_i^{\text{SVL}'}(T) = \beta_i \frac{C_i}{T^2}.$$

Second derivative

$$F_i^{\text{SVL}''}(T) = \beta_i \left(\frac{C_i^2}{T^4} - \frac{2C_i}{T^3} \right).$$

2.2 PD model

The PD model for 1:1 unit-cell mixtures is:

$$\ln\big[4x(1-x)\big] = \frac{2\Delta H_{f,i}}{R} \left(\frac{1}{T_{m,i}} - \frac{1}{T} \right).$$

Define:

$$C_{Ri} = \frac{2\Delta H_{f,i}}{R}, \qquad Q_{Ri}(T) = C_{Ri} \left(\frac{1}{T_{m,i}} - \frac{1}{T} \right), \qquad \beta_{Ri} = e^{Q_{Ri}(T)}.$$

The PD composition is:

$$x_i(T) = \frac{1 - \sqrt{1 - \beta_{Ri}}}{2}, \qquad \gamma_i = \sqrt{1 - \beta_{Ri}}.$$

First derivative

$$F_i^{\text{PD}'}(T) = -\frac{C_{Ri}\beta_{Ri}}{2T^2\gamma_i}.$$

Second derivative

$$F_i^{\text{PD}''}(T) = \frac{C_{Ri}\beta_{Ri}\left[4T(1-\beta_{Ri}) - C_{Ri}(2-\beta_{Ri})\right]}{4T^4(1-\beta_{Ri})^{3/2}}.$$

2.3 Mixed-mode residual

The eutectic condition is:

$$\sum_i X_{\mathrm{eu},i}(T) = 1.$$

Thus the solver residual is:

$$F(T) = \sum_{i\in\mathrm{SVL}} F_i^{\mathrm{SVL}}(T) + \sum_{j\in\mathrm{PD}} F_j^{\mathrm{PD}}(T) - 1.$$

$$F'(T) = \sum_{i\in\mathrm{SVL}} F_i^{\mathrm{SVL}'}(T) + \sum_{j\in\mathrm{PD}} F_j^{\mathrm{PD}'}(T).$$

$$F''(T) = \sum_{i\in\mathrm{SVL}} F_i^{\mathrm{SVL}''}(T) + \sum_{j\in\mathrm{PD}} F_j^{\mathrm{PD}''}(T).$$

2.4 Newton's method

$$T_{k+1} = T_k - \frac{F(T_k)}{F'(T_k)}.$$

2.5 Halley's Method

Halley's update uses first and second derivatives:

$$\boxed{T_{k+1} = T_k - \frac{2F(T_k)F'(T_k)}{2\left[F'(T_k)\right]^2 - F(T_k)F''(T_k)}}$$

2.6 Summary of function residuals

Model	**Residual** $F(T)$	**Derivative** $F'(T)$	**Second Derivative** $F''(T)$
SVL	$\beta_i = e^{Q_i(T)}$	$\beta_i \dfrac{C_i}{T^2}$	$\beta_i \left(\dfrac{C_i^2}{T^4} - \dfrac{2C_i}{T^3} \right)$
PD (composition)	$\dfrac{1 - \sqrt{1 - \beta_{Ri}}}{2}$	$-\dfrac{C_{Ri}\beta_{Ri}}{2T^2\sqrt{1 - \beta_{Ri}}}$	$\dfrac{C_{Ri}\beta_{Ri}\left[4T(1-\beta_{Ri}) - C_{Ri}(2-\beta_{Ri})\right]}{4T^4(1-\beta_{Ri})^{3/2}}$
Mixed residuals	$\sum_i X_{\mathrm{eu},i}(T) - 1$	$\sum_i X'_{\mathrm{eu},i}(T)$	$\sum_i X''_{\mathrm{eu},i}(T)$

3 Selection of Solvers

The MultXeu engine was assessed with four eutectic solvers: Brent, Newton, Halley, and a walk-down (from 1998 VBA-code) method. A bracketing wrapper was recently introduced to improve numerical stability by constraining solver outputs to physically meaningful temperature ranges and providing a fallback to the walk-down solver.

Brent was discarded early due to use of analytical forms and we quickly established it was slower than Newton and Halley. This report summarizes the robustness analysis performed on the remaining three solvers and provides justification for removing the Newton solver from the production codebase.

3.1 Physical background

For any eutectic set of components, the eutectic temperature T_k must lie *below the lowest melting point* of the components participating in that eutectic. The eutectic composition is defined by the root of the residual function

$$f(T) = \sum_i X_i(T) - 1,$$

where $X_i(T)$ is computed using either the PD or SVL model.
A correct solver must:

1. converge to the physically meaningful root of $f(T)$,
2. remain stable under PD stiffness,
3. avoid false roots,
4. produce consistent results across cascades.

3.2 Summary of numerical findings

Halley solver

Across all mixtures tested, Halley:

- converged rapidly,
- agreed with the walk-down solver to numerical precision,
- produced physically meaningful eutectic temperatures,
- remained stable under PD stiffness,
- produced consistent cascade behaviour.

Halley is therefore considered the correct primary solver.

Walk-down solver

The walk-down solver:

- always converged,
- produced correct eutectic temperatures,
- served as a reliable fallback for Halley.

Its only drawback is speed, Halley cuts the time by a third with minimal accuracy sacrifice.

Newton solver

Even when wrapped in a bracketing layer, Newton exhibited multiple failure modes:

- convergence to *false roots* inside the bracket,
- physically incorrect eutectic temperatures despite small residuals,
- incorrect eutectic compositions,
- catastrophic divergence in later cascade nodes,
- sensitivity to initial guess,
- instability in the presence of PD terms.

A representative example is shown below. For a five-component mixture, the correct eutectic temperature (Halley/VBA) was $T_k = 107.23°C$. Bracketed Newton returned $T_k = 119.43°C$, leading to incorrect compositions and a corrupted cascade, including a spurious node at 304°C.

These errors occurred *even when Newton remained inside the bracket and produced a small residual*. This demonstrates that Newton can converge to mathematically valid but physically meaningless roots.

3.3 Conclusion: removal of the Newton solver

The robustness study shows that Newton:

- is not reliable for eutectic computation,
- cannot be made reliable through bracketing,
- produces incorrect results that propagate through cascades,
- offers no advantages over Halley or the walk-down solver.

We therefore we implement the following:

1. removing Newton from the production codebase,
2. retaining it only in a separate benchmarking branch if needed,
3. adopting Halley as the primary solver for CRYSS due to superior speed, and,
4. wrapping Halley in a bracketing layer,
5. using the higher accuracy walk-down solver as default in the NODES app.

4 Composition interpolation between nodes

This document summarizes two core mathematical components of the MultXeu engine:

1. The **gradient–intercept model** used to compute solid composition at arbitrary recovery values between eutectic nodes.
2. The **relative solvent volume model**, which estimates solvent demand based on the relativesolubility behaviour of material removed at each node (nb in this system is assumed to behave according to the ideal approximation).

These models allow MultXeu to generate continuous purification profiles and relative solvent–volume estimates without re-running the full eutectic cascade.

4.1 Definitions

Let the system contain N components with initial mole fractions

$$X^{(0)} = \{X_1^{(0)}, X_2^{(0)}, \ldots, X_N^{(0)}\}.$$

The eutectic cascade produces a sequence of nodes indexed by $k = 1, 2, \ldots, K$, each with:

- Eutectic composition vector $X^{\text{eu}}(k)$,
- Node recovery fraction R_k (mass removed at node k),
- Cumulative recovery remaining after node k:

$$C_k = 1 - \sum_{i=1}^{k} R_i.$$

4.2 Gradient–intercept model

Between nodes k and $k+1$, the composition of the solid varies *linearly* with recovery.

Let R be the overall recovery (fraction of solid remaining), with

$$C_{k+1} \leq R \leq C_k.$$

For each component i, define a linear model:

$$X_i(R) = m_{i,k}\, R + b_{i,k},$$

where:

$$m_{i,k} = \frac{X_i(C_{k+1}) - X_i(C_k)}{C_{k+1} - C_k},$$
$$b_{i,k} = X_i(C_k) - m_{i,k}\, C_k.$$

Thus, the gradient $m_{i,k}$ and intercept $b_{i,k}$ are computed directly from the compositions at the node boundaries.

4.2.1 Node identification

Given a recovery value R, the correct node interval is found by locating k such that:

$$C_{k+1} \leq R \leq C_k.$$

The composition at recovery R is then:

$$X(R) = \{X_1(R), X_2(R), \ldots, X_N(R)\}.$$

4.3 Mass balance

Because the system is closed, liquid composition can be computed from solid composition via:

$$X_i^{\text{liq}}(R) = \frac{X_i^{(0)} - R\, X_i(R)}{1 - R}.$$

This avoids re-running the eutectic solver.

4.4 Relative solvent volume model

The solvent requirement for each node is governed by the **relative solubility** of the material removed (note that we are assuming that a given eutectic has a certain solubility and that we are calculating the solution of that specific eutectic's dissolution; given the n–dimensional eutectic has a lower melting point than the (n-1)–dimensional eutectic, it follows that the (n-1)–dimensional eutectic will have a lower solubility, and so on. Therefore, if we arbitrarily assign 1 volume of solvent at any point, we can specify another solvent volume against this arbitrary point. We are, of course, not calculating the actual solubility, but the *relative* solubility).

4.5 Mass removed at node

Let R_k be the mass fraction removed at node k.

4.6 Mass dissolved at node

The mass dissolved into the mother liquor at node k is:

$$D_k = \sum_{i=1}^{N} \left(X_i^{\text{liq}}(C_{k+1}) - X_i^{\text{liq}}(C_k) \right).$$

4.7 Relative solubility

Define the relative solubility at node k as:

$$S_k = \frac{D_k}{R_k}.$$

This ratio expresses how much material must dissolve per unit mass removed, and therefore reflects the *difficulty* of the purification step.

4.8 Relative solvent volume

The cumulative solvent requirement up to node k is:

$$V_k = \sum_{i=1}^{k} S_i.$$

This quantity increases monotonically and provides a physically meaningful measure of solvent demand across the purification path.

4.9 Summary

- The gradient–intercept model provides continuous composition profiles between eutectic nodes.
- The relative solubility model provides physically meaningful solvent–volume estimates.
- Together, these models enable MultXeu to generate purification profiles, solvent requirements, and realistic CRYSS input files without re-running the full eutectic cascade.

5 Canonical eutectic path and liquor interpolation

We consider a canonical eutectic path discretized into nodes and segments.

5.1 Nodes and solids from gradients and intercepts

Each canonical node m has an associated eutectic composition

$$\mathbf{X}_{\text{eu}}^{(m)} = \left(X_{\text{eu},1}^{(m)}, \ldots, X_{\text{eu},N}^{(m)}\right),$$

obtained from the eutectic model.

Along the path, the solid compositions at the nodes are not arbitrary; they are determined by the canonical gradients and intercepts of the eutectic path. For each component k,

$$X_{\text{s},k}^{(m)} = a_k + b_k\, f_m,$$

where a_k and b_k are the intercept and gradient for component k, and f_m is the canonical path parameter at node m. Thus the solid compositions

$$\mathbf{X}_{\text{s}}^{(m)} = \left(X_{\text{s},1}^{(m)}, \ldots, X_{\text{s},N}^{(m)}\right)$$

are fully determined by the gradient/intercept representation of the eutectic path. These are the X_s values used in the slicing engine.

5.2 Segments and recovery intervals

The path is partitioned into segments indexed by j. Each segment j covers a recovery interval

$$R_{\text{start}}^{(j)} \quad \text{to} \quad R_{\text{end}}^{(j)},$$

with an associated solid composition $\mathbf{X}_{\text{s}}^{(j)}$ (within the segment in the canonical representation).

For each segment we also have the liquor compositions at the segment boundaries:

$$\mathbf{X}_{\text{liq,start}}^{(j)} \quad \text{and} \quad \mathbf{X}_{\text{liq,end}}^{(j)},$$

which encode the cumulative effect of all previous eutectic removals up to $R_{\text{start}}^{(j)}$ and $R_{\text{end}}^{(j)}$, respectively. In other words, the history of "subtracting eutectic" is embedded in these boundary liquors; we do not recompute the full liquor path at slice time.

5.3 Liquor interpolation within a segment

Given a target recovery R lying inside segment j,

$$R_{\text{end}}^{(j)} \le R \le R_{\text{start}}^{(j)},$$

we define a local interpolation parameter

$$t(R) = \frac{R_{\text{start}}^{(j)} - R}{R_{\text{start}}^{(j)} - R_{\text{end}}^{(j)}} \quad \in [0,1].$$

The liquor composition at recovery R is then obtained by linear interpolation between the segment boundary liquors:

$$\mathbf{X}_{\text{liq}}(R) = (1 - t(R))\, \mathbf{X}_{\text{liq,start}}^{(j)} + t(R)\, \mathbf{X}_{\text{liq,end}}^{(j)}.$$

Componentwise, for each $k = 1, \ldots, N$,

$$X_{\mathrm{liq},k}(R) = (1 - t(R))\, X^{(j)}_{\mathrm{liq,start},k} + t(R)\, X^{(j)}_{\mathrm{liq,end},k}.$$

Because both $\mathbf{X}^{(j)}_{\mathrm{liq,start}}$ and $\mathbf{X}^{(j)}_{\mathrm{liq,end}}$ are valid compositions (i.e. each component lies in $[0, 1]$ and the vectors are normalized), the interpolated liquor $\mathbf{X}_{\mathrm{liq}}(R)$ is also bounded and normalized.

5.4 Consistency with the lever rule

At any slice recovery R, the feed composition $\mathbf{X}_{\mathrm{c}}$, solid composition $\mathbf{X}_{\mathrm{s}}$, and liquor composition $\mathbf{X}_{\mathrm{liq}}(R)$ satisfy the lever rule:

$$\mathbf{X}_{\mathrm{c}} = R\, \mathbf{X}_{\mathrm{s}} + (1 - R)\, \mathbf{X}_{\mathrm{liq}}(R),$$

or componentwise,

$$X_{\mathrm{c},k} = R\, X_{\mathrm{s},k} + (1 - R)\, X_{\mathrm{liq},k}(R).$$

Equivalently, for each component k,

$$R = \frac{X_{\mathrm{c},k} - X_{\mathrm{liq},k}(R)}{X_{\mathrm{s},k} - X_{\mathrm{liq},k}(R)},$$

and all components yield the same recovery R (within numerical tolerance). This is exactly the mass-balance check that can be performed in the Excel output.

5.5 Summary

- The solid compositions $\mathbf{X}_{\mathrm{s}}$ used in the slices come directly from the canonical gradients and intercepts of the eutectic path; they are not invented at slice time. - The liquor compositions along the path are represented by their values at segment boundaries, $\mathbf{X}^{(j)}_{\mathrm{liq,start}}$ and $\mathbf{X}^{(j)}_{\mathrm{liq,end}}$, which encode the cumulative effect of eutectic removal. - For any slice recovery R inside a segment, the liquor composition is obtained by linear interpolation between these boundary liquors, ensuring bounded, normalized, and mass-balance-consistent $\mathbf{X}_{\mathrm{liq}}(R)$.

6 Experimental error model

The error model applied to the crystallization mixture data consists of two physically motivated components: (i) cross-contamination error between solid and liquor phases, and (ii) analytical measurement error applied to all composition fields. The model preserves mass balance through explicit renormalization and recomputes the lever-rule recovery parameter R using a robust multi-component strategy.

6.1 Notation

For each component i in a mixture, the canonical compositions are:

$$X_{c,i}, \qquad X_{s,i}, \qquad X_{\ell,i}$$

corresponding to the starting mixture, solid phase, and liquor phase, respectively.

A user–specified error magnitude p (in percent) defines the standard deviation of the Gaussian perturbations.

6.2 Stage 1: cross-contamination error

Cross–contamination models sampling error between phases. A single random Gaussian perturbation is generated per phase:

$$\alpha_s = |\mathcal{N}(0,\, p/100)|\,, \qquad \alpha_\ell = |\mathcal{N}(0,\, p/100)|\,.$$

These perturbations are applied homogeneously across all components:

$$X'_{s,i} = X_{s,i} + \alpha_s X_{\ell,i}, \qquad X'_{\ell,i} = X_{\ell,i} + \alpha_\ell X_{s,i}.$$

The contaminated compositions are renormalized:

$$X'_{s,i} \leftarrow \frac{X'_{s,i}}{\sum_j X'_{s,j}}, \qquad X'_{\ell,i} \leftarrow \frac{X'_{\ell,i}}{\sum_j X'_{\ell,j}}.$$

6.3 Stage 2: analytical error

Analytical error is applied independently to each component of each composition field. For each component i:

$$\epsilon_i = \mathcal{N}(0,\, p/100), \qquad |\epsilon_i| \le 0.02.$$

The perturbed compositions are:

$$X''_{c,i} = X_{c,i}(1+\epsilon_i), \qquad X''_{s,i} = X'_{s,i}(1+\epsilon_i), \qquad X''_{\ell,i} = X'_{\ell,i}(1+\epsilon_i).$$

A second renormalization enforces mass balance:

$$X''_{c,i} \leftarrow \frac{X''_{c,i}}{\sum_j X''_{c,j}}, \qquad X''_{s,i} \leftarrow \frac{X''_{s,i}}{\sum_j X''_{s,j}}, \qquad X''_{\ell,i} \leftarrow \frac{X''_{\ell,i}}{\sum_j X''_{\ell,j}}.$$

6.4 Stage 3: multi-component lever-rule recovery

For each component i, a recovery estimate is computed:

$$R_i = \frac{X''_{c,i} - X''_{\ell,i}}{X''_{s,i} - X''_{\ell,i}}.$$

Components with $|X''_{s,i} - X''_{\ell,i}| < 10^{-12}$ are discarded. Outliers are rejected using median filtering:

$$R_i \text{ retained if } |R_i - \text{median}(R)| < 0.1.$$

The final recovery value is the mean of the filtered set:

$$R = \frac{1}{N_{\text{valid}}} \sum_{\text{valid } i} R_i.$$

Physical constraints are enforced:

$$R \leftarrow \min(0.9999,\, \max(0,\, R)).$$

6.5 Summary

The complete error model therefore consists of:

1. One–way cross–contamination between solid and liquor phases.
2. Independent Gaussian analytical error applied per component.
3. Mass–balance renormalization after each error stage.
4. Robust multi–component lever–rule recovery calculation.
5. Physical clamping of R to the interval [0, 0.9999].

This procedure produces realistic experimental scatter while based on likely error sources from typical laboratory study of a crystallization system.